

medium.com

Math for ML: Understand Regression Decision Trees with Simple Examples

Rayan Yassminh

15–20 minutes

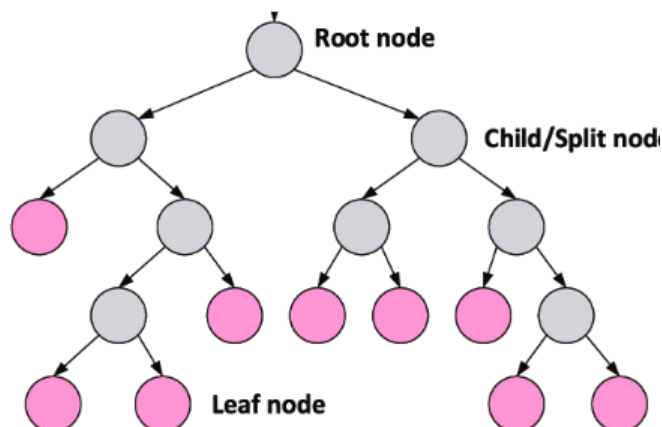


12 min read

Jun 29, 2025

Before diving into advanced machine learning models, it's essential to understand decision trees. They're not just easy to interpret, but they form the backbone of powerful methods like Random Forests and Gradient Boosting. Mastering decision trees means mastering the logic behind many of today's top algorithms.

DECISION TREE



Imagine you're helping a friend decide how much to charge for renting out her apartment.

You start by asking some thoughtful, simple questions:

- Is it in a big city?
- How many bedrooms does it have?
- Is there a balcony?

With each answer, you narrow things down, grouping similar listings. Finally, you suggest a price based on the average of others in that group. That's the heart of a **regression decision tree**: it breaks down decisions by asking one question at a time, creating a clear path from input to prediction.

But what if you kept asking more and more questions?

- Is the microwave new?
- What color are the curtains?

- How many plants are in the living room?

Now the advice gets oddly specific. You've split the data so finely that your friend ends up with a price tailored to just one or two examples, overfitting the past and losing sight of the bigger picture.

This is where tuning and pruning come in.

We **tune** the tree by setting limits, such as the maximum depth, the number of samples required for a split, or the minimum number of listings in a final group. It's like saying, "Let's not go overboard with the questions. Stick to what really matters."

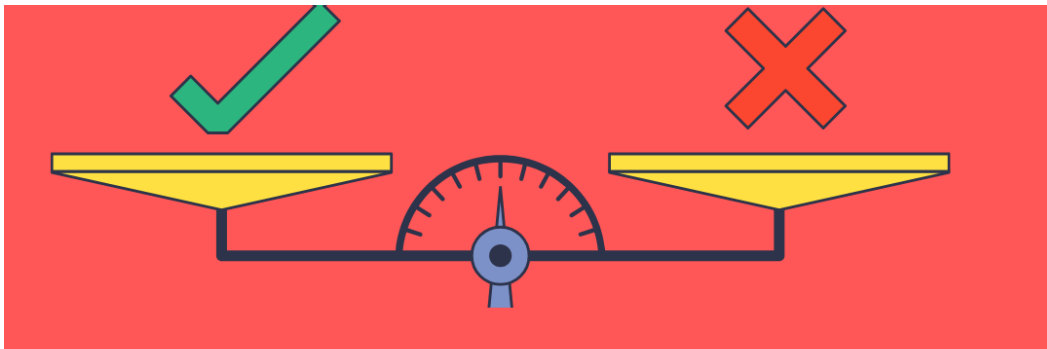
We **prune** the tree by trimming unnecessary branches and removing overly specific rules that don't significantly improve predictions on new apartments. It's like saying, "Let's cut out those weird curtain-color questions — they aren't helpful."

By **tuning** and **pruning**, we help the tree stay bright and focused — just enough detail to be useful, but not so much that it memorizes noise. That way, it gives better predictions not just for your friend's apartment, but for any new one that comes along.

Why choose Regression Decision Tree?

When your target is numeric (house price, temperature, sales ...), a regression tree offers a straightforward, easily-interpretable way to model **"if-then-else"** relationships between features and the value you want to predict. Unlike linear regression, a tree can capture sharp jumps, plateaus, and piece-wise trends without you having to write a single equation.

Press enter or click to view image in full size



Pros	Cons
Handles <i>non-linear</i> data with zero preprocessing	High-depth trees over-fit quickly
Works with mixed numeric / categorical features	Predictions are <i>piece-wise constant</i> (can look jagged)
Easy to explain & visualise	Small training changes can grow a different tree (high variance)
No need to scale features	Often beaten in accuracy by ensembles

Key Mathematical Concepts used in DT

The tree grows by selecting splits that minimize prediction error. The most common methods are:

(a) Mean Squared Error (MSE) Reduction

- Purpose: Measures impurity in a node; the split that minimizes MSE is chosen.

Press enter or click to view image in full size

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y})^2$$

where:

- y_i = true value,
- $\hat{y}$ = predicted value (mean of responses in the node),
- N = number of samples in the node.

Splitting Criterion: Choose the split that maximizes MSE reduction:

$$\Delta\text{MSE} = \text{MSE}_{\text{parent}} - \left(\frac{N_{\text{left}}}{N} \text{MSE}_{\text{left}} + \frac{N_{\text{right}}}{N} \text{MSE}_{\text{right}} \right)$$

(b) Variance Reduction

- Similar to MSE, but computes the variance of target values in child nodes:

Press enter or click to view image in full size

$$\text{Var}(y) = \frac{1}{N} \sum_{i=1}^N (y_i - \bar{y})^2$$

where $\bar{y}$ is the mean of target values in the node.

- The split that maximizes variance reduction is selected.

Prediction in a Leaf Node

- For a given test instance falling into a leaf, the predicted value is the mean of the training samples in that leaf:

$$\hat{y}_{\text{leaf}} = \frac{1}{N_{\text{leaf}}} \sum_{i \in \text{leaf}} y_i$$

Regularization (Preventing Overfitting)

To avoid overly complex trees, regularization techniques are applied:

Maximum Depth: Limits the tree's growth depth.

Minimum Samples per Split: Ensures a node must have at least k samples to split further.

Minimum Samples per Leaf: Ensures that each leaf contains at least m samples.

Pruning (Cost-Complexity Pruning): Removes branches that add little predictive power using:

$$R_{\alpha}(T) = R(T) + \alpha|T|$$

where:

- $R(T)$ = training error,

- $|T|$ = number of leaves,
- α = complexity parameter.

What is the Complexity Parameter α (Alpha)?

When you grow a decision tree, it can easily become **too large**, splitting the data into small groups that fit the training data perfectly but **fail to generalize** to new data. That's called **overfitting**.

To fix this, we use something called the **complexity parameter α (alpha)**.

What does alpha do?

Alpha controls how much we “prune” the tree, removing parts that don't help much.

- If **α is small**, the tree keeps more branches (more complexity).
- If **α is large**, we cut off more branches (simpler tree).

The goal is to find a sweet spot: **a tree that's not too complex, but still accurate.**

How pruning works with α :

1. For each split in the tree, we check if **removing it** would reduce the overall error **after adding a small penalty** for complexity.
2. We try different values of α , ranging from small to large, and **prune the tree step by step**.
3. For each pruned tree, we test its performance on a **validation set**.
4. Finally, we select the α that yields the **best validation score** — that's our optimal balance between accuracy and simplicity.

Think of α as your model's “spring cleaning” budget. The higher the budget, the more branches you're allowed to throw out. However, pruning too much can cause the model to forget important information. Prune too little, and it remembers everything, even the noise.

Handling Missing Values

Imagine a decision tree as a curious guide trying to classify or predict something by asking one question at a time. But what happens if the answer to a question is missing?

Surrogate Splits are the solution in Decision Trees: If the main splitting feature is missing, they use another feature that behaves similarly (correlates well with the missing one).

Handling Categorical Variables

While numerical splits are easy, handling **categorical variables** is trickier:

- You can create one branch per category (if there are a few categories)
- Or encode them using one-hot or ordinal encoding (less ideal for trees unless you do special handling)

Feature Importance Calculation

- The importance of a feature is computed based on how much it reduces MSE across all splits:

The importance Importance_j of feature j is computed as:

$$\text{Importance}_j = \frac{1}{N} \sum_{\text{splits } s \text{ using } j} N_s \cdot \Delta\text{MSE}_s$$

where:

- N = total number of training samples,
- N_s = number of samples at split s ,
- ΔMSE_s = reduction in MSE due to split s .

Where:

Press enter or click to view image in full size

Compute **MSE reduction** due to the split:

$$\Delta\text{MSE} = \text{MSE}_{\text{parent}} - \left(\frac{N_L}{N_p} \text{MSE}_{\text{left}} + \frac{N_R}{N_p} \text{MSE}_{\text{right}} \right)$$

Press enter or click to view image in full size

Sometimes, importance values are normalized to sum to 1 or 100% for interpretability:

$$\text{Normalized Importance}_j = \frac{\text{Importance}_j}{\sum_{k=1}^m \text{Importance}_k}$$

where m = total number of features.

A feature is important if:

- It is used frequently in splits (appears often in the tree).
- It leads to significant MSE reductions (significantly improves prediction accuracy when used).

A feature is unimportant if:

- It is rarely used (appears in a few splits).
- It provides little MSE reduction (doesn't help much in prediction).

Regression Decision Tree step by step

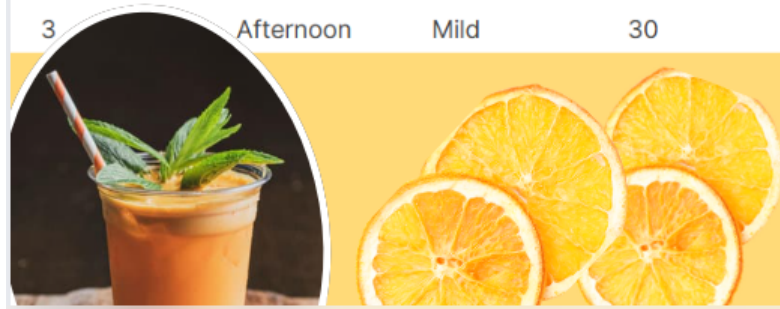
Imagine you run a lemonade stand along a long street. You keep notes to decide tomorrow's stock; you could fit one big straight line through every point, but traffic patterns change from block to block. A decision tree mimics how you would really think:

Step 1: Know Your Goal

You want to **predict a number** — not a label — so you're doing **regression**, not classification.

Goal: Predict how many cups of lemonade you'll sell under certain conditions (like temperature, time of day, etc.).

Dataset Example			
Block	Time	Weather	Sales (Cups)
1	Morning	Hot	35
1	Afternoon	Cold	15
2	Morning	Mild	25
2	Afternoon	Hot	40
3	Morning	Hot	50
3	Afternoon	Mild	30



Step 2 : Start at the Root Node

Press enter or click to view image in full size

- All data: Sales = [35, 15, 25, 40, 50, 30]
- Mean Sales:

$$\bar{y} = \frac{35 + 15 + 25 + 40 + 50 + 30}{6} = 32.5 \text{ cups}$$

- Initial MSE (How wrong the mean is):

$$\text{MSE} = \frac{(35 - 32.5)^2 + (15 - 32.5)^2 + \dots + (30 - 32.5)^2}{6} = 141.67$$

Step 3: Find Best Split (Minimize MSE)

We test every possible split and pick the one that reduces MSE the most.

Press enter or click to view image in full size

Option 1: Split on "Block ≤ 1.5"

- Left (Block ≤ 1.5): Sales = [35, 15] → Mean = 25

$$\text{MSE} = \frac{(35 - 25)^2 + (15 - 25)^2}{2} = 100$$

- Right (Block > 1.5): Sales = [25, 40, 50, 30] → Mean = 36.25

$$\text{MSE} = \frac{(25 - 36.25)^2 + \dots + (30 - 36.25)^2}{4} = 89.06$$

- Total MSE:

$$2 \quad \dots \quad 4 \quad \dots$$

$$\frac{3}{6} \times 100 + \frac{3}{6} \times 89.06 = 92.71$$

Press enter or click to view image in full size

Option 2: Split on "Weather = Hot?"

- **Left (Hot):** Sales = [35, 40, 50] → Mean = 41.67

$$\text{MSE} = \frac{(35 - 41.67)^2 + \dots + (50 - 41.67)^2}{3} = 38.89$$

- **Right (Not Hot):** Sales = [15, 25, 30] → Mean = 23.33

$$\text{MSE} = \frac{(15 - 23.33)^2 + \dots + (30 - 23.33)^2}{3} = 38.89$$

- **Total MSE:**

$$\frac{3}{6} \times 38.89 + \frac{3}{6} \times 38.89 = 38.89$$

Best Split: "Weather = Hot?" gives the lowest MSE (38.89) → Better than "Block ≤ 1.5" (92.71).

Step 4: Split Again (Recursively)

Now, each child node can split further.

Press enter or click to view image in full size

Left Node (Weather = Hot)

- Sales: [35, 40, 50]
- Try splitting on **Block ≤ 2.5**:
 - **Left (Block ≤ 2.5):** [35, 40] → Mean = 37.5

$$\text{MSE} = \frac{(35 - 37.5)^2 + (40 - 37.5)^2}{2} = 6.25$$

- **Right (Block > 2.5):** [50] → MSE = 0
- **Total MSE:**

$$\frac{2}{3} \times 6.25 + \frac{1}{3} \times 0 = 4.17$$

- **Best Split:** "Block ≤ 2.5" → Further divides hot days by location.

Press enter or click to view image in full size

Right Node (Weather ≠ Hot)

- Sales: [15, 25, 30]
- Try splitting on **Time = Morning?**:
 - **Left (Morning):** [25] → MSE = 0
 - **Right (Afternoon):** [15, 30] → Mean = 22.5

$$\text{MSE} = \frac{(15 - 22.5)^2 + (30 - 22.5)^2}{2} = 56.25$$

- **Total MSE:**

$$\frac{1}{3} \times 0 + \frac{2}{3} \times 56.25 = 37.5$$

- **Best Split:** "Time = Morning?" → Divides non-hot days by time.

Final Decision Tree:

1. Is the Weather Hot?
 - Yes →
 2. Is Block ≤ 2.5 ?
 - Yes → Predict 37.5 cups
 - No → Predict 50 cups
 - No →
 3. Is it Morning?
 - Yes → Predict 25 cups
 - No → Predict 22.5 cups



Prediction Example

- Input: Block = 3, Time = Afternoon, Weather = Hot
- Path: Hot? → Yes → Block > 2.5 → Predict 50 cups

Step-by-Step Feature Importance Calculation for the Lemonade Stand Regression Tree

Calculate Total MSE Reduction

From our tree, splits happened at:

1. Root Node → Split on "Weather"
 - MSE before split: 141.67
 - MSE after split: 38.89
 - Reduction: $141.67 - 38.89 = 102.78$

2. Hot Days → Split on “Block”

- MSE before split: 38.89 (only for the 3 hot-day samples)
- MSE after split: 4.17
- Reduction: $38.89 - 4.17 = 34.72$

3. Non-Hot Days → Split on “Time”

- MSE before split: 38.89 (only for the 3 non-hot samples)
- MSE after split: 37.5
- Reduction: $38.89 - 37.5 = 1.39$

Total MSE Reduction:

$$102.78 \text{ (Weather)} + 34.72 \text{ (Block)} + 1.39 \text{ (Time)} = 138.89$$

Assign Importance to Each Feature

Weather

- Used once (root split).
- MSE reduction: 102.78
- Importance: $102.78 / 138.89 = 0.74$ (74%)

Block

- Used once (splitting hot days).
- MSE reduction: 34.72
- Importance: $34.72 / 138.89 = 0.25$ (25%)

Time

- Used once (splitting non-hot days).
- MSE reduction: 1.39
- Importance: $1.39 / 138.89 = 0.01$ (1%)

Final Feature Importance

- The weather is the most critical factor, as it influences sales data into high and low categories.
- Block refines predictions for hot days (e.g., Block 3 sells 50 cups).
- Time barely matters; non-hot days sell just as well.

Step-by-Step Pruning Process Calculation for the Lemonade Stand Regression Tree

Unpruned Tree

```
[Weather = Hot?]
├── Yes → [Block ≤ 2.5?] → Predict 37.5 or 50
└── No → [Time = Morning?] → Predict 25 or 22.5
```

- Total leaves: 4
- MSE (Training): Low (overfits noise in data).

Pruned Tree

```
[Weather = Hot?]
├── Yes → Predict 43.3 (mean of 35, 40, 50)
└── No → Predict 23.3 (mean of 15, 25, 30)
```

- Total leaves: 2 (pruned Block and Time splits).
- MSE (Validation): Better generalization.

Always keep in mind the anatomy of a regression tree!

- **Root node:** where the very first split happens
- **Internal nodes:** further questions ("Is Temperature ≤ 24 °C?")
- **Leaf nodes:** hold a single number: the average target value of training points that landed there
- **Loss:** trees measure "purity" with **mean-squared error (MSE)**; a split is kept if it reduces total MSE the most

Ex1: In this example, we will do the following:

1. Generate a noisy wave.
2. Fit a depth-3 DecisionTreeRegressor.
3. Visualize its step-wise prediction.

Observe how the model flattens sections where the sine curve is smooth and jumps where it bends sharply.

In this example, we will learn about under-fitting vs over-fitting

The following three figures compare depths 2, 5, and "no limit".

- **Shallow (depth 2)** — only broad plateaus → under-fits
- **Moderate (depth 5)** — follows the main wiggles nicely
- **Unlimited depth** — chases every noise blip → over-fits

Tune `max_depth`, `min_samples_leaf`, or `ccp_alpha` (cost-complexity pruning) to find the optimal setting, typically using cross-validation.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeRegressor, plot_tree
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

rng = np.random.RandomState(1)
X = np.sort(5 * rng.rand(80, 1), axis=0)
y = np.sin(X).ravel()
y[::5] += 0.5 - rng.rand(16)

tree_simple = DecisionTreeRegressor(max_depth=3, random_state=42)
tree_simple.fit(X, y)

X_test = np.arange(0.0, 5.0, 0.01)[:, np.newaxis]
```

```

y_pred = tree_simple.predict(X_test)

plt.figure()
plt.scatter(X, y, s=25, label="Training samples")
plt.plot(X_test, y_pred, linewidth=2, label="Tree prediction")
plt.xlabel("X")
plt.ylabel("y")
plt.title("Depth-3 Decision Tree Regression")
plt.legend()
plt.show()

plt.figure(figsize=(10, 6))
plot_tree(tree_simple, filled=True, rounded=True,
          feature_names=["X"], max_depth=3)
plt.title("Structure of the depth-3 regression tree")
plt.show()

depths = [2, 5, None]
models = []
for d in depths:
    reg = DecisionTreeRegressor(max_depth=d, random_state=42)
    reg.fit(X, y)
    models.append(reg)

    y_pred = reg.predict(X_test)
    plt.figure()
    plt.scatter(X, y, s=20)
    plt.plot(X_test, y_pred, linewidth=2)
    plt.xlabel("X")
    plt.ylabel("y")
    title = f"Decision Tree (max_depth={d if d else 'None'})"
    plt.title(title)
    plt.show()

```

Ex2: A real multivariate case (Diabetes dataset)

The final code cell trains an 8-level tree on 10 medical variables.

- **RMSE $\approx$ 55** indicates the error in the original disease progression units.
- The bar chart (last figure) ranks feature importance — here, **BMI** dominates the splits, followed by blood-serum marker **S5**.

Because decision trees split on one feature at a time, the importance is intuitive: a higher bar means it is more often chosen for high-impact splits.

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeRegressor, plot_tree
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

data = load_diabetes()
X_diab, y_diab = data.data, data.target

X_tr, X_te, y_tr, y_te = train_test_split(X_diab, y_diab,
                                          test_size=0.2,

```

```
random_state=42)

tree_diab = DecisionTreeRegressor(max_depth=8,
min_samples_leaf=30,
                                random_state=42)
tree_diab.fit(X_tr, y_tr)

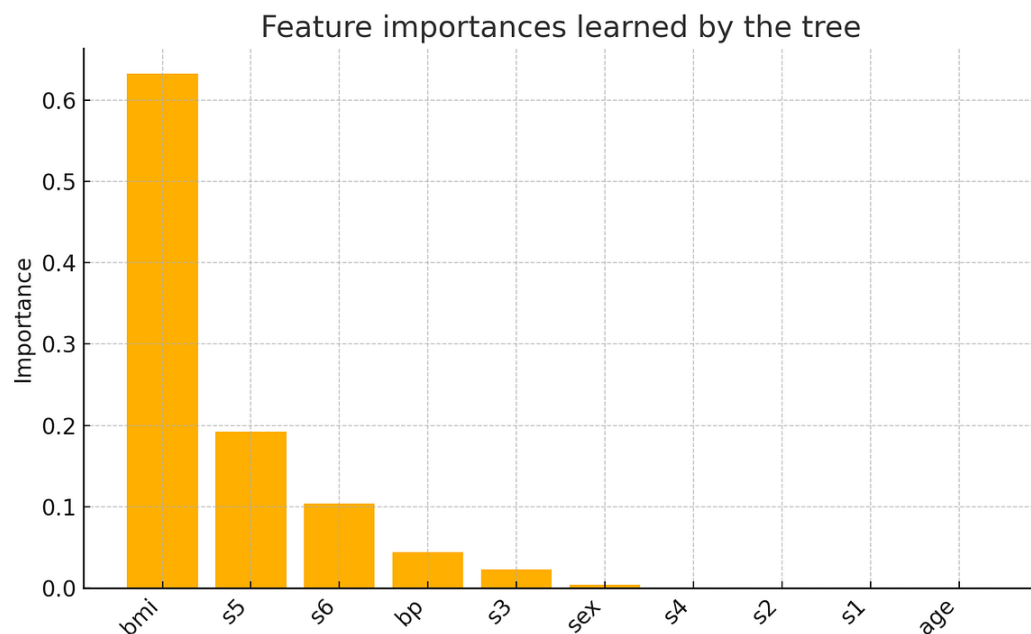
mse = mean_squared_error(y_te, tree_diab.predict(X_te))
rmse = np.sqrt(mse)
r2_score = tree_diab.score(X_te, y_te)
print(f"Diabetes regression RMSE (depth=8): {rmse:.3f}")
print(f"Diabetes r2_score (depth=8): {r2_score:.3f}")

plt.figure(figsize=(8, 5))
order = np.argsort(tree_diab.feature_importances_)[::-1]
plt.bar(range(len(order)), tree_diab.feature_importances_[order])
plt.xticks(range(len(order)), np.array(data.feature_names)[order],
           rotation=45, ha="right")
plt.ylabel("Importance")
plt.title("Feature importances learned by the tree")
plt.tight_layout()
plt.show()
```

Diabetes regression RMSE (depth=8): 54.717

Diabetes r2_score (depth=8): 0.435

Press enter or click to view image in full size

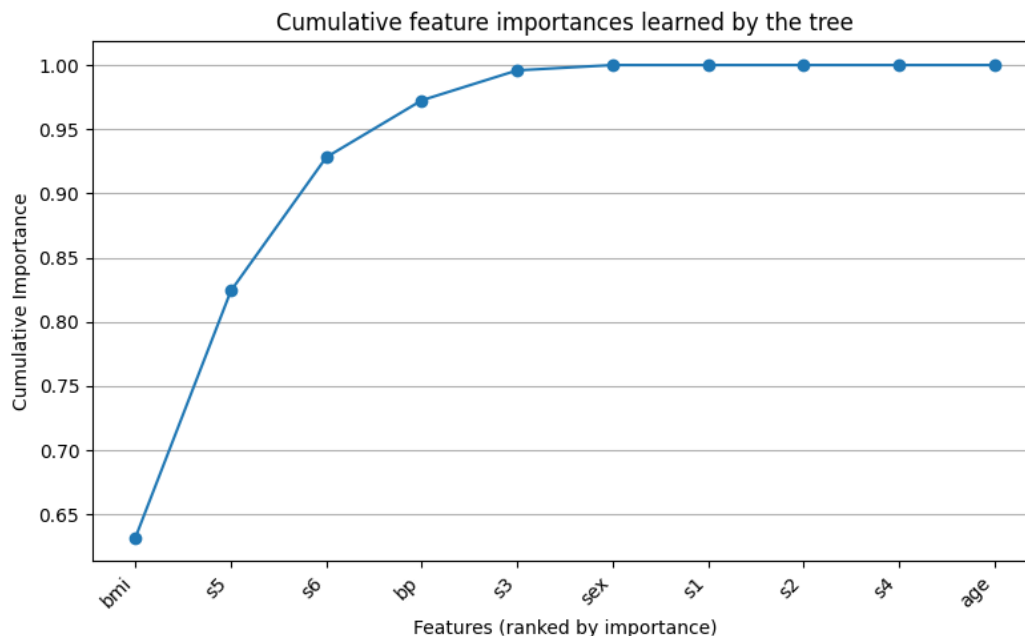


```
sorted_indices = np.argsort(tree_diab.feature_importances_)[::-1]
sorted_importances =
tree_diab.feature_importances_[sorted_indices]
cumulative_importances = np.cumsum(sorted_importances)

plt.figure(figsize=(8, 5))
plt.plot(range(len(cumulative_importances)),
cumulative_importances, marker='o', linestyle='-')
```

```
plt.xticks(range(len(sorted_indices)),
np.array(data.feature_names)[sorted_indices], rotation=45,
ha="right")
plt.ylabel("Cumulative Importance")
plt.xlabel("Features (ranked by importance)")
plt.title("Cumulative feature importances learned by the tree")
plt.grid(axis='y')
plt.tight_layout()
plt.show()
```

Press enter or click to view image in full size



For example, the first feature (BMI) alone accounts for about 0.63 of the total importance. Including the top 2 features (BMI and S5) accounts for around 0.82 of the total importance. A steep initial rise followed by a flattening curve indicates that a few features are very important, while many others contribute little. You can use this plot to identify a subset of features that capture most of the predictive power, which can be useful for feature selection or simplifying the model.

Here's the updated analysis using the diabetes dataset, showing how the model's performance (R^2 score) changes as you adjust the pruning complexity parameter α . This helps you find a good balance between model simplicity and predictive power.

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
import matplotlib.pyplot as plt
import pandas as pd
```

```
data = load_diabetes()
X = data.data
y = data.target
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)
```

```

tree = DecisionTreeRegressor(random_state=42)
path = tree.cost_complexity_pruning_path(X_train, y_train)

ccp_alphas = path.ccp_alphas[:-1]
impurities = path.impurities[:-1]

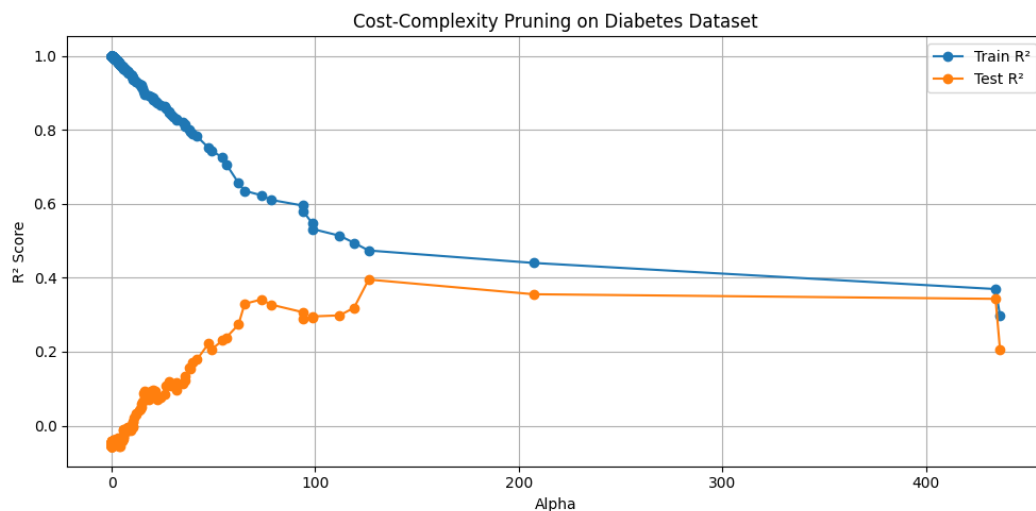
trees = []
train_scores = []
test_scores = []
for alpha in ccp_alphas:
    model = DecisionTreeRegressor(random_state=42,
    ccp_alpha=alpha)
    model.fit(X_train, y_train)
    trees.append(model)
    train_scores.append(model.score(X_train, y_train))
    test_scores.append(model.score(X_test, y_test))

results_df = pd.DataFrame({
    'alpha': ccp_alphas,
    'train_r2': train_scores,
    'test_r2': test_scores
})

plt.figure(figsize=(10, 5))
plt.plot(ccp_alphas, train_scores, marker='o', label='Train R2')
plt.plot(ccp_alphas, test_scores, marker='o', label='Test R2')
plt.xlabel("Alpha")
plt.ylabel("R2 Score")
plt.title("Cost-Complexity Pruning on Diabetes Dataset")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

```

Press enter or click to view image in full size



```

from sklearn.tree import plot_tree
import numpy as np

best_index = np.argmax(test_scores)
best_alpha = ccp_alphas[best_index]

```

```
best_model = trees[best_index]
```

```
best_train_r2 = train_scores[best_index]
```

```
best_test_r2 = test_scores[best_index]
```

```
plt.figure(figsize=(20, 10))
```

```
plot_tree(best_model, feature_names=data.feature_names,
```

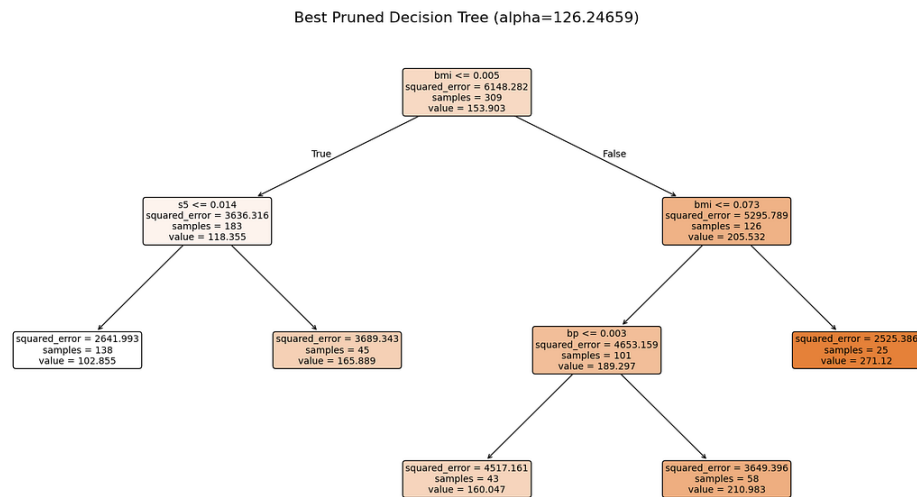
```
filled=True, rounded=True, fontsize=10)
```

```
plt.title(f"Best Pruned Decision Tree (alpha={best_alpha:.5f})",  
          fontsize=16)
```

```
plt.show()
```

```
{  
    "best_alpha": best_alpha,  
    "train_r2": best_train_r2,  
    "test_r2": best_test_r2  
}
```

Press enter or click to view image in full size



The visual above illustrates the structure of the best-pruned decision tree. This version balances complexity and performance better than deeper, overfitted trees.